AppGuard AI

Frontend Integration Guide

YellowSense Technologies · RBI HaRBInger 2025

API Base URL — http://34.14.189.124:8000 · Swagger UI: http://34.14.189.124:8000/docs

Requirements Overview

#	Requirement	Priority	Breaking?	Status
1	Separate Static vs Final Score label	CRITICAL	No	Pending
2	Fix Citizen Portal API (Consumer Tab)	CRITICAL	YES	Pending
3	PDF Download Button	Done	—	COMPLETE
4	Fix "None" engine label	Medium	No	Pending
5	Flagged domains now return objects	HIGH	YES	Pending
6	Cartel Graph node classification & colours	Medium	No	Pending
7	Wire Alerts Feed	Optional	No	Pending
8	Dynamic KFS compliance card	Medium	No	Pending
9	Fix cached app sandbox timing	HIGH	No	Pending

■ *Requirements marked YES under Breaking? will cause existing UI to malfunction if not updated. Do REQ 2 and REQ 5 first.*

REQ 1 Separate Static Score vs Final Combined Score — CRITICAL

The risk gauge currently shows one number with no distinction between Stage 1 (static analysis only) and Stage 2 (sandbox enriched final score). Judges need to see these as two separate things.

During Stage 1 (sandbox still running)

- Label the gauge: **"Static Analysis Score"**
- Show subtext: *"Deep sandbox analysis in progress..."*
- Sandbox status box must read: **DETONATING_IN_BACKGROUND**

After Stage 2 completes (final_dynamic_verdict received)

- If **final_dynamic_verdict === "CRITICAL"** → set gauge to 100, colour RED
- If **final_dynamic_verdict === "MAINTAIN_STATIC_SCORE"** → keep gauge value as-is
- Change gauge label to **"Final Combined Score"**
- Change subtext to **"Analysis Complete"**
- Sandbox status box must now read: **COMPLETED**

Key rule: The sandbox status box value must be driven by whether Stage 2 data is loaded in React state — NOT by the dynamic_sandbox_status field in the Stage 1 response. See REQ 9 for the related timing fix.

```
// WRONG — shows COMPLETED before Stage 2 data is loaded in state
const statusDisplay = stage1Response.dynamic_sandbox_status

// CORRECT — driven by state
const statusDisplay = stage2Data ? "COMPLETED" : "DETONATING_IN_BACKGROUND"
```

REQ 2 Fix Citizen Portal API Call (Consumer Tab) — CRITICAL

This is in the **Consumer Tab (Tab 3)** — specifically the Citizen Verification Portal search bar on the left panel. The current API call points to the wrong endpoint and will always fail. Must be fixed before the demo.

Change the API call:

```
// OLD — wrong endpoint for a package ID search
POST /api/v1/analyze/unified   (multipart/form-data with APK file)

// NEW — correct endpoint
POST /api/v1/analyze/check
Content-Type: application/json
Body: { "package_id": "com.example.app" }
```

Response fields to render:

```
{
  "app_name":           string   // App title to display
  "verdict":            "LOW" | "MEDIUM" | "HIGH"
  "risk_score":         float    // Multiply by 100 for % display
  "found_on_playstore": boolean  // false → show Ghost App red warning
  "flagged_reasons":    string[] // Show as bullet list
  "in_dla_registry":    boolean  // true → show green "RBI Verified" badge
  "message":            string   // Summary paragraph
}
```

Result card colours:

Condition	Card	Message to show
verdict: LOW	Green	Verified — App is in RBI authorised registry.
verdict: MEDIUM	Yellow	Caution — App has compliance gaps. Verify before installing.
verdict: HIGH	Red	WARNING — App is NOT authorised. Do not install.
found_on_playstore: false	Red	Ghost App — Not on Play Store. Extremely high risk.
in_dla_registry: true	Green badge	Show "RBI Verified" badge on result card

REQ 3 PDF Download Button — COMPLETE

Already implemented. No action needed. The button calls GET /api/v1/report/pdf/{package_id} and opens the PDF in a new tab.

REQ 4 Fix "None" Engine Label — MEDIUM

In the Analysis Engines Fired tag list, when an app is sideloaded, the second engine tag currently shows a raw string that looks broken. Replace it with a proper label.

```
// If engine string contains "None" or "not found", replace with:
"Ghost App Detection Engine"

// Detection rule:
const displayEngine = (engine) =>
  engine.includes("None") || engine.includes("not found")
    ? "Ghost App Detection Engine"
    : engine
```

REQ 5 Flagged Domains Are Now Objects — BREAKING

Breaking change. The flagged_burner_domains field in the Stage 2 dynamic report has changed from an array of plain strings to an array of objects. Any code that renders this field will break without this update.

Old format (no longer valid):

```
["loc.map.baidu.com", "frontloan-api.co"]
```

New format:

```
[
  {
    "domain":         "loc.map.baidu.com",
    "classification": "CHINESE_INFRASTRUCTURE",
    "risk":           "HIGH"
  },
  {
    "domain":         "frontloan-api.co",
    "classification": "BURNER_DOMAIN",
    "risk":           "HIGH"
  }
]
```

Update your render code:

```
// OLD — will break
domains.map(domain => <span>{domain}</span>)

// NEW — use item.domain
domains.map(item => <span>{item.domain}</span>)
```

Add a Classification column to the network domains table:

classification value	Badge colour	Meaning
CHINESE_INFRASTRUCTURE	Red #C0392B	Routes data to Chinese state-linked servers (Baidu, Xiaomi, Tencent)
BURNER_DOMAIN	Orange #D35400	Uses disposable/fraud TLD (.xyz .cc .su .top)
STAGING_IN_PRODUCTION	Amber #9A7D0A	App hits a test/staging API with real user data — compliance violation
SUSPICIOUS	Dark orange	Unknown domain with no legitimate business reason

---

REQ 6 Cartel Graph Node Classification & Colours

MEDIUM

The cartel_graph_data.json has been rebuilt with clean, noise-free data. All legitimate infrastructure domains (Google, GitHub, Adjust, AWS) have been removed. Only genuinely suspicious domains remain. The graph now carries classification metadata so the frontend can colour-code nodes meaningfully.

Node data structure:

```
// APK node — represents a fake app
{
  "id":         "com.from.outside",
  "type":       "apk",
  "label":      "fakeapp1.apk",
  "verdict":    "HIGH",
  "risk_score": 1.0          // float 0.0–1.0
}

// Domain node — represents a suspicious server the APK called
{
  "id":             "loc.map.baidu.com",
  "type":           "domain",
  "label":          "loc.map.baidu.com",
  "classification": "CHINESE_INFRASTRUCTURE"
}

// Edge — APK communicated with this domain during sandbox execution
{ "source": "com.from.outside", "target": "loc.map.baidu.com" }
```

Visual rules for APK nodes (type: "apk"):

Property	Value
Shape	Circle or hexagon
Colour	Navy blue #0A1628
Size	Proportional to risk_score — e.g. base 50px × risk_score (min 30px)
Label	Show the label field (e.g. fakeapp1.apk)
Tooltip	Show verdict + risk score as percentage on hover

Visual rules for Domain nodes (type: "domain"):

classification	Colour	Hex	Shape
CHINESE_INFRASTRUCTURE	Red	#C0392B	Diamond
BURNER_DOMAIN	Orange	#D35400	Diamond
STAGING_IN_PRODUCTION	Amber	#9A7D0A	Diamond
SUSPICIOUS	Dark orange-red	#C0392B	Diamond

Edge rules:

- Directed arrow: APK node → Domain node
- Edge colour: match the target domain node colour
- Animated edges recommended for demo visual impact

React Flow implementation example:

```
const getNodeColor = (node) => {
  if (node.type === 'apk') return '#0A1628'
  const map = {
    'CHINESE_INFRASTRUCTURE': '#C0392B',
    'BURNER_DOMAIN':          '#D35400',
    'STAGING_IN_PRODUCTION':  '#9A7D0A',
    'SUSPICIOUS':             '#C0392B',
  }
  return map[node.classification] || '#C0392B'
}

const rfNodes = graphData.nodes.map(n => ({
  id:   n.id,
  data: { label: n.label },
  style: {
    background: getNodeColor(n),
    color:      'white',
    border:     'none',
    borderRadius: n.type === 'apk' ? '50%' : '4px',
    width:  n.type === 'apk' ? Math.max(30, (n.risk_score || 0.5) * 60) : 44,
    height: n.type === 'apk' ? Math.max(30, (n.risk_score || 0.5) * 60) : 44,
    fontSize: 7,
  }
}))

const rfEdges = graphData.edges.map((e, i) => ({
  id: `e${i}`, source: e.source, target: e.target, animated: true,
  style: { stroke: '#C0392B' }
}))
```

REQ 7 Wire Alerts Feed — OPTIONAL

The alerts endpoint now returns real live data from all previously analyzed HIGH/CRITICAL apps. Wire to any 'Recent Detections' or 'Live Alerts' section in the B2B dashboard.

```
GET /api/v1/alerts
```

Response structure:

```
{
  "total_high_risk_detections": number,
  "alerts": [
```

```
    {
      "package_id":            string,
      "app_name":              string,
      "developer":             string,
      "verdict":               "HIGH" | "CRITICAL",
      "risk_score":            float,
      "top_flag":              string,   // Most important flag text
      "permission_violations": number,   // Count of RBI violations
      "report_url":            string,   // Link to dynamic report endpoint
      "pdf_url":               string    // Link to PDF download endpoint
    }
  ]
}
```

Recommended: refresh every 30 seconds with setInterval to make the dashboard feel live during the demo.

REQ 8 Dynamic KFS Compliance Card

MEDIUM

The Stage 2 dynamic report now includes a dynamic_kfs_compliance object showing whether the app displayed mandatory RBI financial disclosures when it was actually running inside the sandbox. Show as a compliance checklist card in the Stage 2 panel.

Data location in Stage 2 response:

```
{
  "dynamic_kfs_compliance": {
    "compliant":     boolean,
    "signals_found": number,      // 0 to 4
    "signals_total": 4,
    "signals": {
      "apr_shown_at_runtime":  boolean,
      "kfs_screen_detected":   boolean,
      "grievance_shown":       boolean,
      "cooling_off_mentioned": boolean
    },
    "verdict": string
  }
}
```

Checklist card — one row per signal:

Signal key	What it means	Pass / Fail
apr_shown_at_runtime	App showed APR / interest rate during sandbox run	✓ / ✗
kfs_screen_detected	KFS or sanction letter screen appeared at runtime	✓ / ✗
grievance_shown	Grievance Redressal Officer info visible at runtime	✓ / ✗
cooling_off_mentioned	Cooling-off / loan cancellation option shown	✓ / ✗

Show summary line: "X / 4 RBI disclosures detected at runtime" — green if signals_found >= 2, red if signals_found < 2. Card title: "RBI KFS Runtime Compliance Check".

REQ 9 Fix Cached App Sandbox Timing

HIGH

When an app has been analyzed before (both static and dynamic reports cached), Stage 1 immediately returns dynamic_sandbox_status: "COMPLETED". But Stage 2 still shows loading for 10-15 seconds because the frontend waits for the next polling interval tick instead of fetching the already-available report.

Fix 1 — Skip polling for cached apps, fetch immediately:

```
const analyzeApp = async (file) => {
  const stage1 = await submitAPK(file)
  setStage1Data(stage1)

  if (stage1.dynamic_sandbox_status === "COMPLETED") {
    // Cache hit — dynamic report already exists, fetch it now
    const dynamic = await fetchDynamicReport(stage1.package_id)
    setStage2Data(dynamic)
  } else {
    // Fresh analysis — start polling every 10 seconds
    startPolling(stage1.package_id)
  }
}
```

Fix 2 — Drive sandbox status display from React state:

```
// WRONG — shows COMPLETED immediately from Stage 1 field
const statusDisplay = stage1Data.dynamic_sandbox_status

// CORRECT — only shows COMPLETED when Stage 2 data is loaded
const statusDisplay = stage2Data ? "COMPLETED" : "DETONATING_IN_BACKGROUND"
```

Test both scenarios after implementing: (1) Upload a fresh APK never analyzed before — polling should work normally. (2) Re-upload an already analyzed APK — Stage 2 should appear within 2 seconds with no loading state.